

DevOps With AWS Mini Project

Project Title: Containerizing and Deploying the Weather App Using Docker and AWS (ECR, ECS, Fargate)

Student Name : Devendra Ghanekar

Batch Code : Jun2026-A529

Institute : IT Vedant

Course : DevOps With AWS

Submission Date :

Project Overview

This project was completed as part of the ITVedant DevOps Fundamentals course (Docker module). The goal was to get hands-on practice with Docker containerization: installing Docker, writing Dockerfiles, building images, running containers, deploying a multi-container application with Docker Compose, and finally deploying the application on AWS using Amazon ECR, ECS and Fargate.

The project was done with my own Weather App (a frontend and a backend service) instead of the sample OpsMate service. The same steps and tasks of the project brief were followed for this application.

The project covers Docker installation and daemon management, Dockerfile creation, image build and container management, Docker networking and volumes, Docker Compose, AWS container deployment, troubleshooting and professional technical documentation. The work simulates the daily tasks of a Junior DevOps Engineer who containerizes and ships an application.

Project Objectives

- Understand Docker, containers and the difference between virtual machines and containers.
- Install and configure Docker Engine on an Ubuntu Linux server (EC2).
- Manage the Docker daemon and Docker service.
- Create Dockerfiles and build custom Docker images for the Weather App.
- Run, stop, restart, inspect and remove Docker containers.
- Configure Docker networking, port mapping and persistent volumes.
- Deploy a multi-container application using Docker Compose.
- Push Docker images to Amazon Elastic Container Registry (ECR).
- Create an ECS cluster, task definition and service using AWS Fargate.
- Verify the application from a browser and update it with a new image version.
- Monitor, troubleshoot and document the Docker implementation professionally.

Task 1: Understanding Containerization

Virtualization vs Containerization

Virtualization runs many full operating systems on one machine using a hypervisor. Containerization runs many isolated applications on one operating system kernel. Containers are lighter, start in seconds and use fewer resources.

Feature	Virtual Machine	Docker Container
Operating system	Full guest OS in every VM	Shares the host OS kernel
Size	Several GB	Few MB to few hundred MB
Startup time	Minutes	Seconds
Resource usage	High	Low
Isolation	Strong (hardware level)	Process level
Portability	Limited	Runs the same everywhere

Advantages of Containers

- Same environment in development, testing and production.
- Fast start and stop; easy to scale.
- Efficient use of CPU and memory.
- Easy to version, share and roll back using images.
- Works well with CI/CD pipelines.

Docker Architecture

Component	Description
Docker Client	The docker command line tool that sends commands to the daemon.
Docker Host / Daemon (dockerd)	The background service that builds images and runs containers.
Docker Engine	Docker client + daemon + REST API working together.
Docker Image	Read-only template that contains the app, runtime and dependencies.
Docker Container	Running instance of an image.
Docker Registry	Storage for images: Docker Hub (public) or Amazon ECR (private).

Components Needed to Containerize the Weather App

- Frontend service: React/Vite application served using Nginx.
- Backend service: Node.js API that fetches weather data.
- Database service: for stored data (used in Docker Compose).
- Dockerfile for each service, plus compose.yaml to run them together.
- Amazon ECR to store images and ECS Fargate to run them.

Task 2: Installing Docker and Managing Docker Daemon

Launch and Connect to Ubuntu EC2

An Ubuntu EC2 instance was launched in the AWS console with ports 22 (SSH) and the application ports allowed in the security group. The instance was accessed using SSH.

```
ssh -i "C:\Users\Deva\OneDrive\Pictures\Downloads\deva.pem" ubuntu@
```

Install Docker

```
sudo apt update -y
sudo apt upgrade -y
sudo apt install docker.io -y
```

Manage the Docker Service

```
sudo systemctl status docker
sudo systemctl start docker
sudo systemctl stop docker
sudo systemctl restart docker
sudo systemctl enable docker
```

Run Docker Without sudo

```
sudo usermod -aG docker $USER
newgrp docker
```

Verify Installation

```
docker --version
docker info
docker ps
docker images
systemctl status docker
docker run hello-world
```

Task 3: Creating Dockerfile for the Weather App

The application runtime (Node.js) and its dependencies (listed in package.json) were identified. A project directory was created and a Dockerfile was written for each service.

```
mkdir Weather-Docker-Project && cd Weather-Docker-Project
```

Backend Dockerfile (Node.js)

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5000

CMD ["npm", "start"]
```

Frontend Dockerfile (Multi-stage: Vite build + Nginx)

```
# Build stage

FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN chmod +x node_modules/.bin/*

RUN npm run build

# Production stage

FROM nginx:alpine

COPY nginx.conf /etc/nginx/conf.d/default.conf

COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Dockerfile Instructions Explained

Instruction	Purpose
FROM	Selects the base image (small alpine images keep the size low).
WORKDIR	Sets the working folder inside the container.
COPY	Copies application files into the image.
RUN	Runs commands during build, e.g. npm install.
EXPOSE	Documents the port used by the application.

Instruction	Purpose
CMD	Default command to start the application.

Note: The ports and file names above are the ones used in my application; they can differ in your code. Update them as per your project.

Task 4: Building Docker Images and Running Containers

Build and Verify Images

```
docker build -t weather-back:v1 ./backend
docker build -t weather-front:v1 ./frontend
docker images
docker inspect weather-back:v1
```

```
ubuntu@ip-172-31-40-244:~/weather-app$ docker images
```

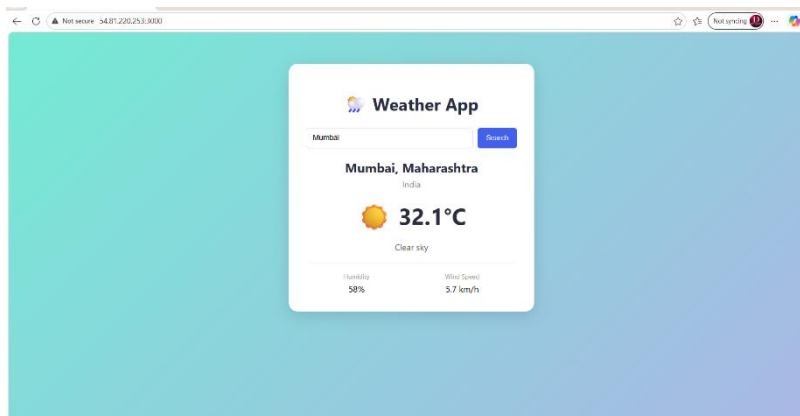
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
back-img:v1	03a5321fdefa	210MB	51.1MB	U
front-img:v1	fee8ef7cedb5	93.8MB	26.4MB	U
nginx:alpine	df221db836e1	94.4MB	27.2MB	
node:20-alpine	fb4cd12c85ee	194MB	48.8MB	
postgres:16-alpine	721873c34ceb	420MB	117MB	

```
ubuntu@ip-172-31-40-244:~/weather-app$
```

Run Containers with Port Mapping

```
docker run -d --name weather-back -p 5000:5000 back-img:v1
docker run -d --name weather-front -p 80:80 front-img:v1
docker ps
```

The Weather App was opened in the browser using <http://<EC2-PUBLIC-IP>> and worked successfully.



Manage Containers

```
docker logs weather-back
docker inspect weather-back
docker stop weather-back
docker start weather-back
docker restart weather-back
docker rm -f weather-back weather-front
docker rmi weather-back:v1
docker system prune -f
```

Command	Use
docker build	Creates an image from a Dockerfile
docker images	Lists local images
docker run	Starts a container from an image
docker ps	Lists running containers
docker logs	Shows container logs

Command	Use
docker inspect	Shows detailed configuration
docker stop / start / restart	Controls the container state
docker rm / rmi	Removes containers / images

Task 5: Multi-Container Deployment Using Docker Compose

A multi-container version of the Weather App was created with three services: frontend, backend and database. The services talk to each other through a custom Docker network and the database data is stored in a named volume.

compose.yaml

```
services:

  backend:
    image: back-img:v1
    container_name: weather-backend
    ports:
      - "5000:5000"
    restart: unless-stopped

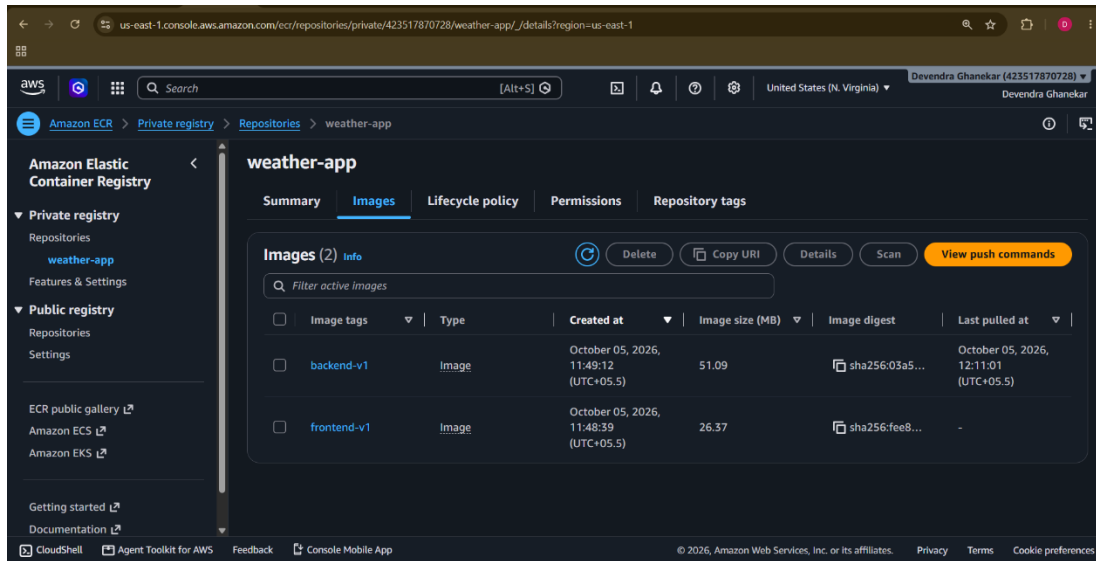
  frontend:
    image: front-img:v1
    container_name: weather-frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
    restart: unless-stopped
```

Run and Manage the Stack

```
docker compose up -d --build
docker compose ps
docker compose logs
docker compose logs backend
docker compose stop
docker compose start
docker compose down
```

Task 6: Deploying on AWS Using ECR, ECS and Fargate

Amazon ECR



Amazon ECS with Fargate

- Created an ECS cluster (weather-cluster) with AWS Fargate as the compute option.
- Created a Task Definition (Fargate, 0.5 vCPU, 1 GB memory) and added the container using the ECR image URI.
- Set the container port, environment variables and CloudWatch logging (awslogs driver).
- Created an ECS Service with desired task count = 1.
- Selected VPC, public subnets and a security group allowing the application port (inbound).
- Enabled public IP and deployed the service on Fargate.

Application Verification

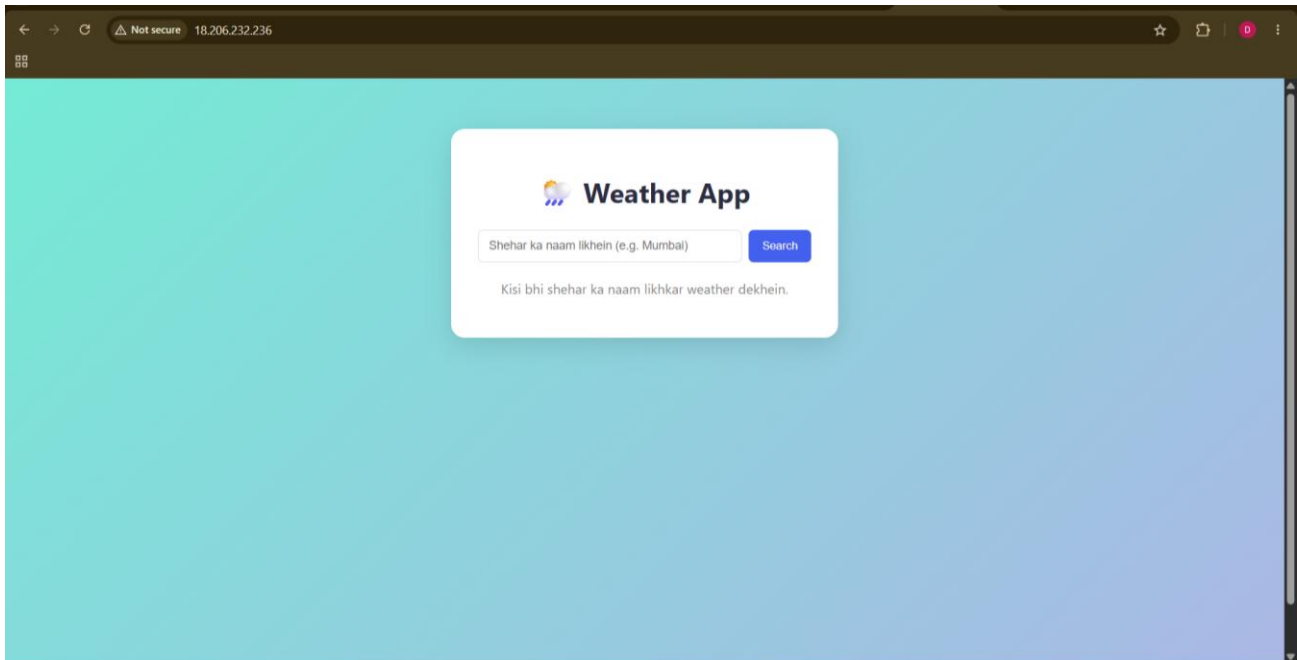
- ECS task status was RUNNING and the service status was ACTIVE.
- Container logs were checked in CloudWatch Logs.
- The public IP of the task was opened in the browser and the Weather App worked.

Update the Application

A small change was made in the application, a new image (v2) was built, pushed to ECR and deployed to ECS by creating a new task definition revision and updating the service (force new deployment).

```
docker build -t weather-front:v2 ./frontend
docker tag weather-front:v2 <ACCOUNT-ID>.dkr.ecr.us-east-1.amazonaws.com/weather-front:v2
docker push <ACCOUNT-ID>.dkr.ecr.us-east-1.amazonaws.com/weather-front:v2
aws ecs update-service --cluster weather-cluster --service weather-service --force-new-deployment
```

Final architecture: Developer → Dockerfile → Docker Image → Amazon ECR → ECS Task Definition → ECS Service → AWS Fargate → Weather App



Task 7: Monitoring, Troubleshooting & Documentation

The Docker implementation was analyzed. The details were also saved in docker-deployment-report.txt.

Area	Details / Command
Docker installation	docker --version, hello-world ran successfully
Docker daemon status	systemctl status docker: active (running)
Image information	docker images, docker inspect
Container information	docker ps, docker inspect
Resource usage	docker stats --no-stream
Application logs	docker logs / docker compose logs
Compose service status	docker compose ps
ECR image information	Repository weather-front / weather-back, tag v1 and v2
ECS service status	Service ACTIVE, desired count 1 = running count 1
Fargate task status	Task RUNNING, public IP reachable

Security Considerations

- Do not store passwords in the Dockerfile; use environment variables or AWS Secrets Manager.
- Add .env to .dockerignore and .gitignore.
- Use small, trusted base images (alpine) and scan images (ECR scan on push).
- Run containers as a non-root user where possible.
- Open only the required ports in security groups; keep SSH limited to my IP.
- Use IAM roles with least privilege.

Resource Optimization Suggestions

- Use multi-stage builds and alpine images to reduce image size.
- Choose the smallest Fargate CPU/memory that works.
- Use ECR lifecycle policies to delete old images.
- Set a log retention period in CloudWatch.
- Stop or delete AWS resources after evaluation to avoid charges.

Improvements Suggested

- Add an Application Load Balancer in front of ECS tasks and use HTTPS.
- Automate build and push using a CI/CD pipeline (Jenkins from my previous project).
- Use Amazon RDS instead of a database container in production.
- Add health checks and auto scaling for the ECS service.

Repository Folder Structure

```
Weather-Docker-Project/
|-- README.md
|-- docker-compose.yml
|
|--weather-app/
|   |-- frontend/
|   |-- backend/
|
|-- scripts/
|   |-- install-docker.sh
|   |-- build-image.sh
|   |-- run-container.sh
|   |-- cleanup.sh
|
|-- screenshots/
|   |-- docker-installation.png
|   |-- docker-daemon.png
|   |-- docker-image.png
|   |-- docker-container.png
|   |-- application-running.png
|   |-- docker-compose.png
|   |-- ecr-repository.png
|   |-- ecr-image.png
|   |-- ecs-cluster.png
|   |-- ecs-task.png
|   |-- fargate-application.png
|
|-- documentation/
|   |-- Docker-Architecture.png
|   |-- Project_Report.pdf
```

A README.md file was prepared covering: Project Overview, Docker Architecture, Installation Steps, Dockerfile Explanation, Compose Setup, AWS Deployment Steps, Commands Used, Folder Structure, Challenges Faced and Learning Outcomes.

Challenges Faced

- ECS task stopped immediately: fixed by checking CloudWatch logs and correcting the container port and environment variables.
- Application not opening from the browser on Fargate: fixed by allowing the port in the security group and enabling public IP.
- ECR push failed with authorization error: fixed by logging in again using `aws ecr get-login-password`.

Conclusion

This project gave me practical experience in Docker containerization and AWS container deployment. I installed and managed Docker on Ubuntu, wrote Dockerfiles, built images, ran and managed containers, deployed a multi-container stack with Docker Compose, and deployed the Weather App on AWS using ECR, ECS and Fargate. I also updated the application with a new image version.

The project helped me understand how containers fit into a real-world DevOps workflow and strengthened my skills for a Junior DevOps Engineer role.